



Encontro 3

Branches e Repositórios Remotos

0:00–0:20

Revisão E2

0:20–1:20

Branches

1:20–1:50

Merge e conflitos

1:50–2:40

fetch/pull/push

2:40–3:00

Quiz (10 perguntas)

```
git checkout -b minha-feature
```

Revisão rápida — Encontro 2

git log e inspeção

Navegar no histórico com log, diff e show.

```
$ git log --oneline --graph --all
```

.gitignore

Não commitar node_modules/, .env e similares.

```
$ git rm --cached <arquivo>
```

git revert (seguro)

Novo commit que desfaz — preserva o histórico compartilhado.

```
$ git revert <hash>
```

git reset --hard (perigo)

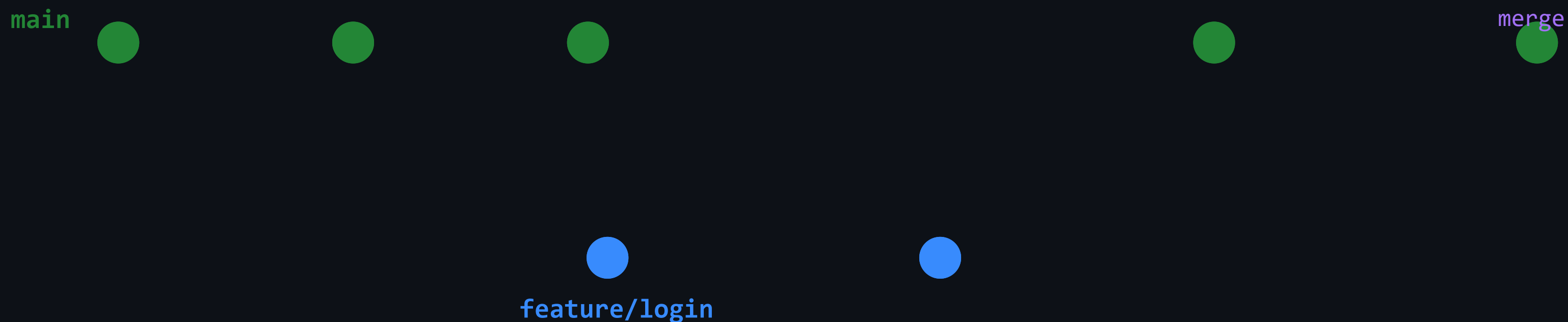
Apaga commit e alterações. Sem volta. Só localmente.

```
$ git reset --hard HEAD~1
```



O que é uma branch e por que usar?

Analogia: branch é um universo paralelo do projeto. Você experimenta sem medo — o original fica intacto na main.



Isola o trabalho

Desenvolva features sem quebrar a main.

Trabalho em equipe

Cada dev na sua branch, sem conflitos diários.

Experimente sem medo

Branches são baratas. Não gostou — deleta.

main sempre estável

Só recebe código revisado e funcionando.

Criando e navegando entre branches

```
$ git branch
```

Listar branches locais

Branch atual aparece com * e em verde.

```
$ git branch nome
```

Criar uma nova branch

Cria mas NÃO muda para ela.

```
$ git checkout nome
```

Mudar para branch existente

Forma clássica — ainda muito usada.

```
$ git checkout -b nome
```

Criar E mudar — atalho do dia a dia

O mais usado. Cria e já entra na branch.

```
$ git switch nome
```

Forma moderna: mudar de branch

Introduzida no Git 2.23. Mais clara.

```
$ git switch -c nome
```

Forma moderna: criar e mudar

Equivale ao checkout -b. Sintaxe mais legível.

Gerenciando branches — deletar e inspecionar

```
$ git branch -d nome
```

Deletar branch (seguro)

Só deleta se a branch já foi mergeada. Protege contra perda de trabalho.

```
$ git branch -D nome
```

Forçar delete

Deleta sem verificar merge. Use para descartar branches inteiras.

```
$ git branch -a
```

Listar locais e remotas

Remotas aparecem como remotes/origin/nome. Útil após git fetch.

```
$ git branch -v
```

Listar com último commit

Mostra hash e mensagem do último commit de cada branch.

```
$ git branch --merged
```

Listar branches já mergeadas

Branches seguras para deletar — trabalho já está na main.

Merge — unindo branches

Fast-forward merge

A main não avançou desde que a branch foi criada. O ponteiro simplesmente avança para o último commit da feature.



```
$ git checkout main  
$ git merge feature/login
```

3-way merge commit

A main avançou desde que a branch foi criada. O Git cria um commit especial que junta as duas linhas.



```
$ git checkout main && git merge feature/login
```

Passo a passo:

1. `git checkout main`
2. `git merge feature/login`
3. Resolver conflitos (se houver)
4. `git push`



Resolução de conflitos de merge

Ocorre quando dois commits editam a mesma linha do mesmo arquivo

arquivo com conflito

```
function saudacao() {
```

```
<<<<<<< HEAD
```

```
    return "Ola, mundo!";
```

```
=====
```

```
    return "Hello, world!";
```

```
>>>>>>> feature/i18n
```

```
}
```

```
<<<<<<< HEAD
```

início — mudança da branch atual (main)

```
=====
```

separador entre as duas versões

```
>>>>>>> feature/i18n
```

fim — mudança da branch mergeada

Como resolver:

1

Editar o arquivo

apague os marcadores e escolha o conteúdo

2

git add arquivo

marcar como resolvido

3

git commit

finalizar o merge

Exercício — provocar e resolver um conflito

Prática em duplas — causar conflito de propósito e resolvê-lo

1

Cada aluno cria sua branch

A: feature/versao-a · B: feature/versao-b

```
$ git checkout -b feature/versao-a
```

2

Editar a mesma linha do README

Cada um escreve um texto diferente na linha 1 e faz commit.

```
$ git add . && git commit -m "Minha versao"
```

3

Aluno A mergeia primeiro

Volta para main e mergeia feature/versao-a. Sem conflito.

```
$ git checkout main && git merge feature/versao-a
```

4

Mergear versao-b — CONFLITO!

O Git reclama que as mesmas linhas foram editadas.

```
$ git merge feature/versao-b
```

5

Resolver e finalizar

Editar o arquivo, remover marcadores, add e commit.

```
$ git add README.md && git commit
```




Repositórios remotos — clone, fetch, pull, push

```
$ git clone <url>
```

Baixar repositório do zero

Cópia local completa com todo histórico. Configura origin automaticamente. Use uma vez por projeto.

```
$ git fetch
```

Baixar atualizações SEM aplicar

Traz commits do remote mas NÃO toca no working directory. Seguro — nunca causa conflito. Inspecione antes de aplicar.

```
$ git pull
```

Baixar E aplicar (fetch + merge)

Atualiza sua branch local. Pode causar conflito se você também editou os mesmos arquivos.

```
$ git push
```

Enviar commits para o remote

Publica seus commits no GitHub. Se o remote tiver commits novos, faça git pull antes.

fetch vs pull — qual usar?

git fetch

Baixa sem aplicar

- Traz commits para o cache local
- Não toca no working directory
- Nunca causa conflito
- Inspeção: `git log origin/main`
- Decida: `git merge origin/main`
- Ideal em equipes ativas

git pull

Baixa e aplica (fetch + merge)

- Equivale a fetch + merge
- Atualiza a branch local direto
- Pode causar conflito
- Use quando ainda não editou nada
- Atalho rápido para projetos solo
- Ideal no início da sessão

Ciclo completo de trabalho em equipe

O fluxo que você vai repetir todos os dias em um trabalho de tecnologia

1

Começar o dia

Sempre sincronize antes de começar. Pega o trabalho do time.

```
$ git pull
```

2

Criar branch da feature

Nunca trabalhe direto na main. Cada tarefa tem sua branch.

```
$ git checkout -b feature/nome
```

3

Commitar o progresso

Commits pequenos e frequentes. Mais fácil revisar e reverter.

```
$ git add . && git commit -m "msg"
```

4

Enviar para o GitHub

Publica a branch para o time ver e abrir pull request.

```
$ git push -u origin feature/nome
```

5

Mergear na main

Integrar o trabalho concluído. Em equipe: via pull request.

```
$ git checkout main && git merge ...
```

Resumo dos comandos — Encontro 3

Branches

```
git branch
```

listar branches

```
git branch nome
```

criar branch

```
git checkout -b nome
```

criar e entrar

```
git switch -c nome
```

criar e entrar (moderno)

```
git branch -d nome
```

deletar mergeada

```
git branch -D nome
```

forçar delete

Merge e conflitos

```
git merge nome
```

mergeia na branch atual

```
git log --graph
```

ver histórico de merges

```
git add <arq>
```

conflito resolvido

```
git merge --abort
```

cancelar merge

Remotos

```
git clone <url>
```

clonar repositório

```
git fetch
```

baixar sem aplicar

```
git pull
```

baixar e aplicar

```
git push
```

enviar commits

```
git push -u origin nome
```

enviar branch nova

```
git remote -v
```

listar remotos

```
git commit -m "Encontro 3 concluído!"
```


ENCONTRO 03

FINALIZADO!



O QUE APRENDEMOS HOJE:



O que são **branches** e por que usar



Criar, navegar e gerenciar **branches**



Merge e resolução de **conflitos**



Repositórios remotos: **fetch**, **pull** e **push**



PRÓXIMO ENCONTRO:

GITHUB NA PRÁTICA
E PROJETO FINAL



Muito bom trabalho!

Até o próximo encontro!

```
git commit -m "Encontro 3 concluído!"
```